A PROOF

A.1 Proof of Lemma 2

We consider the three cases based on the partial-order relationship between v_{ref} and v.

Case 1: $v_{ref} \rightsquigarrow v$ (v must follow v_{ref}). Since e_{ref} occurs before e_v, and each of the mid_slots($v_{ref} \rightsquigarrow v$) events that lie strictly between them must occupy a distinct time instant, we have $t_v - t_{ref} \geq$ mid_slots($v_{ref} \rightsquigarrow v$) + 1 = $\Delta_{\min}(v)$. For the upper bound, let $t_{\min}$ and $t_{\max}$ be the earliest and latest timestamps in M. The window constraint gives $t_{\max} - t_{\min} \leq tw - 1$. Each of the pre_slots($v_{ref}$) events that must occur before e_{ref} forces $t_{\min} \leq t_{ref} -$ pre_slots(v_{ref}). Similarly, each of the post_slots(v) events after e_v forces $t_{\max} \geq t_v +$ post_slots(v). Substituting into the window inequality yields

$$(t_v + \text{post_slots}(v)) - (t_{ref} - \text{pre_slots}(v_{ref})) \leq tw - 1,$$

which rearranges to

$$t_v - t_{ref} \leq (tw - 1) - \text{pre_slots}(v_{ref}) - \text{post_slots}(v) = \Delta_{\max}(v).$$

Case 2: $v \rightsquigarrow v_{ref}$ (v must precede v_{ref}). Symmetric to Case 1. The upper bound follows from the ordering constraint: $t_v - t_{ref} \leq -\text{mid_slots}(v \rightsquigarrow v_{ref}) - 1 = \Delta_{\max}(v)$. The lower bound is obtained from the window constraint by considering the extreme positions of e_v and e_{ref} relative to the other mandatory events, leading to $\Delta_{\min}(v) = -(tw - 1) + \text{post_slots}(v_{ref}) + \text{pre_slots}(v)$.

Case 3: Unordered (no path between v_{ref} and v). Both bounds derive from the window constraint. The maximum possible separation occurs when e_v is the earliest event and e_{ref} the latest, or vice versa, yielding the formulas stated above.

A.2 Proof of Lemma 3

From Lemma 2 we have $\Delta_{\min}(v) \leq t_v - t_{ref} \leq \Delta_{\max}(v)$. Since $t_{ref} \in [e_{ref}.lower, e_{ref}.upper]$, the necessary bounds on t_v are

$$e_{ref}.lower + \Delta_{\min}(v) \;\leq\; t_v \;\leq\; e_{ref}.upper + \Delta_{\max}(v).$$

Combined with $t_v \in [e.lower, e.upper]$, we obtain

$$t_v \in [\ \max(e_v.lower,\ e_{ref}.lower + \Delta_{\min}(v)),$$
$$\min(e_v.upper,\ e_{ref}.upper + \Delta_{\max}(v))\ ],$$

which is exactly $[MinBound(e_v, e_{ref}),\ MaxBound(e_v, e_{ref})]$ by Definition 3. If $[e_v.lower,\ e_v.upper] \cap [MinBound,\ MaxBound] = \emptyset$, then no possible value of t_v can satisfy both constraints, so no valid match containing e_{ref} and e_v can exist.

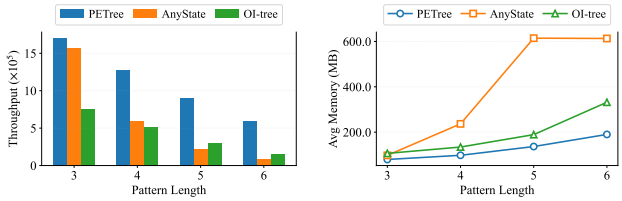
B ADDITIONAL RESULTS

B.1 Effect of Pattern Length with a Growing Window

The pattern-length experiment in the main text uses a fixed window of $tw = 10$. To alleviate the scarcity of complete matches that a fixed window imposes on longer patterns, we repeat the experiment with the window set to twice the pattern length, i.e., $tw = 6, 8, 10, 12$ for lengths 3 to 6, with $\delta = 5$.

Figure 13 reports the results. With a growing window, throughput declines monotonically for all methods, since a larger window admits more complete matches and increases the candidate set within each context. PETree retains the highest throughput and the lowest memory at every length.

This contrasts with the fixed-window setting, where PETree's throughput stabilizes and slightly recovers at length 6, since in that experiment the number of complete matches decreases with pattern length and reduces the total confidence computation effort. The two settings produce different trends for the same sequence of pattern lengths, indicating that throughput is determined jointly by pattern length and window size: increasing the pattern length raises the matching cost per context, while a larger window increases the number of complete matches and the confidence computation effort. The overall trend is a combination of these two effects.



(a) Throughput (b) Average memory usage (in MB)

Figure 13: Evaluation on pattern length with a growing window

B.2 SEQ Queries on the ICU and Activity Datasets

For each dataset we include two pure SEQ queries so that all methods can be compared.

ICU Clinical Data:

Q11 : SEQ($HeartRateHigh, OxygenLow, BPDrop$),

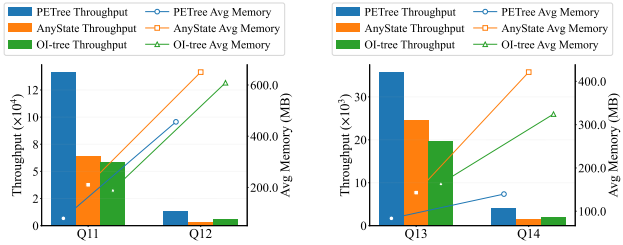
Q12 : SEQ($HeartRateHigh, OxygenLow, BPDrop, BPRise$),

Personal Activity Data:

Q13 : SEQ($Walking, Standing, Sitting$),

Q14 : SEQ($Walking, Standing, Sitting, Lying$),

Figure 14 reports throughput and memory for PETree, AnyState, and OI-tree. In both datasets, PETree achieves the highest throughput and the lowest memory, mirroring the traffic experiment.



(a) ICU clinical data (b) Personal activity data

Figure 14: Performance on ICU and personal activity datasets

C DISCUSSION

Throughput versus arrival rates. Real-world event rates vary widely across applications: A traffic loop sensor and an ICU monitor each produce one event per 5 minutes (about 3×10^{-3} events/s), a wearable sensor samples at 5 Hz (5 events/s), and a modern vehicle may emit thousands of events per second when its on-board sensors are discretized into primitive events. The aggregate event rate of a deployment is the sum over all its sensors.

For the traffic queries, PETree sustains over 10^6 events/s, exceeding the output of any realistic number of loop sensors by several orders of magnitude. For the most complex activity query on the wearable sensor dataset (Q10), throughput is 1.67×10^3 events/s, corresponding to roughly 330 wearable sensors. PETree is therefore not the bottleneck in any of the single-stream settings studied in this paper. However, a single vehicle emitting several thousand events per second already reaches a rate comparable to the Q10 throughput, and deployments that aggregate many such sources (e.g., a fleet of vehicles) under deeply nested patterns could make throughput a constraint. Parallelization across contexts is a natural direction for future work.